Project Overview

The Smart Traffic Grid Simulation is a low-level, text-based x86 Assembly console application modeling a single 4-way intersection (North, East, South, and West). Moving away from traditional static-timer systems, it implements adaptive, queue-driven traffic logic where green light phases adjust dynamically based on real-time vehicle demand.

Core Architecture & Technical Implementation

- Finite State Machine (FSM): Controls the traffic light sequence across a strict 4-phase cycle ($North \rightarrow East \rightarrow South \rightarrow West$) with absolute mutual exclusion to prevent conflicting traffic paths.
- Register Indirect Addressing (ESI): Utilizes index registers (specifically ESI) to dynamically target and modify lane memory variables ([esi]) without hardcoding memory addresses during runtime arithmetic (INC/DEC).
- Non-Blocking Input Handling (INT 16h): Continuously polls BIOS keyboard services (INT 16h, Function 01h) so users can inject manual traffic (N, E, S, W) or exit the program without freezing the simulation loop.

Dynamic Smart Logic & Timer Stability Rule

- Adaptive Variable Timing: Green light durations extend or shorten based on active queue loads rather than fixed preset intervals. If a lane clears early or has a heavy backlog, the FSM reacts fluidly.
- Zero-Flicker Timer Display Rule: To prevent visual clutter and screen stutter on the console, countdown timers remain completely static during active phases. They update and refresh only under two conditions:
    1. *At Phase Initialization:* When a new phase begins and the dynamic time slice is freshly calculated and set.
    2. *At Expiration:* When the counter hits 0, immediately signaling a state transition.

Text-Mode User Interface (TUI / GUI)

- Structured Window Layout: Renders a clean, professional text-mode dashboard reminiscent of classic low-level development environments (such as Turbo C++ or Norton Commander).
- Live Dashboard Elements: Displays real-time signal states (RED, $GREEN$, $YELLOW$), active vehicle counters, hexadecimal ESI memory addresses, and an active phase manager grid without relying on high-level graphical window managers.

Persistent File Logging (TRAFFIC.LOG)

Upon exiting the program, low-level DOS file management interrupts (INT 21h) automatically write a streamlined session audit trail to disk containing precisely four key metrics:

1. Total cars processed from each direction.
2. Max queue length for each individual lane.
3. Global max queue length across the entire intersection during the run.
4. Total manual injections (keypress count via INT 16h).

Project Overview

Modular Breakdown

1. Module 1: User Interface & TUI Dashboard
    a. Renders a structured, windowed text-mode dashboard (TUI) using box-drawing characters to display live signal states, active vehicle counters, and stable countdown timers.
    b. Displays a bottom command bar for manual keyboard controls, automated vehicle status, and log file paths (`TRAFFIC.LOG`).
    c. Polls non-blocking BIOS keyboard services (`INT 16h`) to capture real-time manual vehicle injections (`N`, `E`, `S`, `W`) and exit commands.
2. Module 2: State Machine & Adaptive Timing Control
    a. Executes the core 4-phase sequential traffic light cycle (North→East→South→West) with strict mutual exclusion.

    b. Evaluates real-time lane queue loads via the Phase Manager component to dynamically adjust green light durations instead of using fixed preset intervals.

    c. Enforces the zero-flicker display rule, keeping countdown timers static during active phases and updating them only at initialization or expiration.

3. Module 3: Data & Vehicle Flow Management
    a. Allocates scalar memory variables within the `.DATA` segment to track vehicle queues for all four directions.
    b. Implements register indirect memory addressing via the `ESI` register (displayed directly in the interface's lane panels) to dynamically route target addresses to active lane variables.
    c. Executes low-level assembly arithmetic (`INC`/`DEC`) for automated arrivals and departures.
4. Module 4: File Logging & Persistence
    a. Interfaces with low-level DOS file management interrupts (`INT 21h`) upon program termination.
    b. Exports a streamlined session summary report to `TRAFFIC.LOG` containing the four core metrics: total cars processed per lane, max queue length per lane, global max queue length, and total manual injections.